

TAGD-Progetto-02

Filippo Visconti

1 Introduzione

L'obiettivo di questo progetto è analizzare le dinamiche del controllo di concorrenza e i livelli di isolamento delle transazioni in un DBMS reale. Per garantire l'isolamento e la riproducibilità dell'ambiente di test, l'infrastruttura è stata implementata tramite container. Come DBMS è stato scelto *PostgreSQL*, in esecuzione tramite Docker, e per la simulazione delle transazioni concorrenti un programma scritto in Python, su un Apple MacBook Pro 16 con processore Apple M1 Pro e 16 GB di RAM.

Considerato che Postgres utilizza il Multi-Version Concurrency Control, le operazioni di lettura non bloccano mai le scritture e viceversa. Ogni transazione interroga uno snapshot del database. Questo approccio fa sì che l'implementazione reale dei livelli di isolamento si discosti parzialmente dallo standard. Nello specifico:

- Il livello `READ UNCOMMITTED` non è implementato ed è trattato internamente come `READ COMMITTED`.
- Il livello `REPEATABLE READ` è più rigido rispetto allo standard, arrivando a prevenire anche le letture fantasma (Phantom Reads).
- Il livello `SERIALIZABLE` è gestito tramite Serializable Snapshot Isolation, che garantisce l'isolamento risolvendo i cicli di dipendenza tramite l'aborto di una delle transazioni coinvolte.

Questi comportamenti sono stati dimostrati empiricamente attraverso 6 scenari di test.

2 Esperimenti e Analisi dei Risultati

Di seguito vengono illustrati gli esperimenti eseguiti e l'analisi dei risultati.

2.1 Scenario 1: Lost Update

Questo scenario prevede due transazioni concorrenti che leggono un dato e tentano di incrementarlo.

Eseguendo il test in livello `READ COMMITTED`, entrambe le transazioni leggono il medesimo valore iniziale e procedono all'aggiornamento, portando alla perdita di una delle due modifiche.

```
1 [LOST_UPDATE] --- INIZIO TEST LOST UPDATE (READ COMMITTED) ---
2 [LOST_UPDATE] Tx2 ha letto count=1
3 [LOST_UPDATE] Tx1 ha letto count=1
4 [LOST_UPDATE] Tx1 imposta count a 2
5 [LOST_UPDATE] Tx1 COMMIT confermato.
6 [LOST_UPDATE] Tx2 imposta count a 2
7 [LOST_UPDATE] Tx2 COMMIT confermato.
```

Alzando il livello a `REPEATABLE READ`, i log evidenziano un comportamento differente: la prima transazione che esegue il commit ha successo, mentre la seconda fallisce. Postgres, in questo livello di isolamento, genera un errore di serializzazione se rileva che la riga target dell'aggiornamento è stata modificata da un'altra transazione concorrente dopo lo scatto dello snapshot iniziale, al fine di mantenere l'integrità.

```
1 [LOST_UPDATE] --- INIZIO TEST LOST UPDATE (REPEATABLE READ) ---
2 [LOST_UPDATE] Tx1 ha letto count=1
3 [LOST_UPDATE] Tx2 ha letto count=1
4 [LOST_UPDATE] Tx1 imposta count a 2
5 [LOST_UPDATE] Tx1 COMMIT confermato.
6 [LOST_UPDATE] CONCLUSIONE: Tx2 ABORTITA per Serialization Failure.
```

2.2 Scenario 2: Lettura Inconsistente

In questo test, una transazione legge lo stesso record due volte a distanza di tempo, mentre una seconda transazione lo modifica ed effettua il commit nell'intervallo tra le due letture.

Nel livello di default `READ COMMITTED`, i risultati mostrano che la prima transazione legge due valori differenti. Lo snapshot viene infatti aggiornato all'inizio di ogni singola query.

```

1 [NON_REPEATABLE_READ] --- INIZIO TEST NON-REPEATABLE READ (READ COMMITTED) ---
2 [NON_REPEATABLE_READ] T1 count prima: 2
3 [NON_REPEATABLE_READ] T2: Valore modificato.
4 [NON_REPEATABLE_READ] T2: Valore committato con successo.
5 [NON_REPEATABLE_READ] T1 count dopo: 3
6 [NON_REPEATABLE_READ] Non-Repeatable Read AVVENUTO! (Il valore è cambiato)

```

Impostando il livello `REPEATABLE READ`, la seconda query della prima transazione continua a restituire il valore letto la prima volta, ignorando il commit effettuato dalla seconda transazione. Questo conferma che lo snapshot viene mantenuto coerente e inalterato per l'intera durata della transazione.

```

1 [NON_REPEATABLE_READ] --- INIZIO TEST NON-REPEATABLE READ (REPEATABLE READ) ---
2 [NON_REPEATABLE_READ] T1 count prima: 2
3 [NON_REPEATABLE_READ] T2: Valore modificato.
4 [NON_REPEATABLE_READ] T2: Valore committato con successo.
5 [NON_REPEATABLE_READ] T1 count dopo: 2
6 [NON_REPEATABLE_READ] Non-Repeatable Read PREVENUTO! (I valori coincidono)

```

2.3 Scenario 3: Inserimento Fantasma

In questo test, la transazione T1 esegue un'operazione di aggregazione. Nel frattempo, T2 inserisce un nuovo impiegato che soddisfa tale condizione.

In `READ COMMITTED`, T1 fa una query e trova 2 impiegati. T2 inserisce un nuovo impiegato ('Bianchi') e fa commit. Quando T1 riesegue la query, vede 3 impiegati. È apparso un record "fantasma".

```

1 [PHANTOM_READ] --- INIZIO TEST PHANTOM READ (READ COMMITTED) ---
2 [PHANTOM_READ] T1 (Prima lettura): Trovati 2 impiegati.
3 [PHANTOM_READ] T2: Nuovo impiegato 'Bianchi' inserito con successo.
4 [PHANTOM_READ] T1 (Seconda lettura): Trovati 3 impiegati.
5 [PHANTOM_READ] Phantom Read AVVENUTO!

```

In teoria, il livello `REPEATABLE READ` consente gli inserimenti fantasma. Tuttavia, i log dell'esperimento dimostrano che in Postgres questa anomalia non si verifica. Infatti T1 interroga uno snapshot creato all'inizio della transazione che rimane immutabile, isolandola dai nuovi inserimenti di T2. Di fatto, in Postgres, il livello `REPEATABLE READ` offre una protezione superiore rispetto allo standard.

```

1 [PHANTOM_READ] --- INIZIO TEST PHANTOM READ (REPEATABLE READ) ---
2 [PHANTOM_READ] T1 (Prima lettura): Trovati 2 impiegati.
3 [PHANTOM_READ] T2: Nuovo impiegato 'Bianchi' inserito con successo.
4 [PHANTOM_READ] T1 (Seconda lettura): Trovati 2 impiegati.
5 [PHANTOM_READ] Phantom Read PREVENUTO!

```

2.4 Scenario 4: Write Skew

Questo scenario analizza due transazioni concorrenti che leggono il numero totale degli impiegati per poi inserire un nuovo record basato su tale conteggio.

In `REPEATABLE READ`, entrambe le transazioni leggono `count=2`. Ognuna procede con il proprio aggiornamento e il proprio commit. Nessuna delle due tocca le righe modificate dall'altra, quindi il DBMS non rileva conflitti diretti a livello di record e permette a entrambe di concludere con successo, lasciando passare l'anomalia.

```

1 [WRITE_SKEW] --- INIZIO TEST WRITE SKEW (Livello: REPEATABLE_READ) ---
2 [WRITE_SKEW] Tx2 ha letto count = 2
3 [WRITE_SKEW] Tx1 ha letto count = 2
4 [WRITE_SKEW] Tx1 ha letto updated count = 3
5 [WRITE_SKEW] Tx2 ha letto updated count = 3
6 [WRITE_SKEW] Tx1 ha letto post commit count = 4
7 [WRITE_SKEW] CONCLUSIONE: Tx1 COMMIT completato con successo.
8 [WRITE_SKEW] Tx2 ha letto post commit count = 4
9 [WRITE_SKEW] CONCLUSIONE: Tx2 COMMIT completato con successo.

```

Impostando l'isolamento a livello `SERIALIZABLE`, il sistema garantisce un'esecuzione serializzabile e monitora le dipendenze in lettura e scrittura e, al rilevamento di un ciclo di dipendenze (entrambe hanno preso decisioni di scrittura basandosi su dati che l'altra stava modificando) che violerebbe la serializzabilità, lascia procedere la prima transazione ed esegue l'aborto forzato della seconda al momento del commit, restituendo un'eccezione di *Serialization Failure*.

```

1 [WRITE_SKEW] --- INIZIO TEST WRITE SKEW (Livello: SERIALIZABLE) ---
2 [WRITE_SKEW] Tx2 ha letto count = 2
3 [WRITE_SKEW] Tx1 ha letto count = 2
4 [WRITE_SKEW] Tx2 ha letto updated count = 3
5 [WRITE_SKEW] Tx1 ha letto updated count = 3
6 [WRITE_SKEW] Tx1 ABORTITA per Serialization Failure
7 [WRITE_SKEW] Tx2 ha letto post commit count = 3
8 [WRITE_SKEW] CONCLUSIONE: Tx2 COMMIT completato con successo.

```

2.5 Scenario 5: Deadlock

Questo test simula un'attesa circolare: T1 blocca il record "Rossi" e attende "Bruni", mentre T2 blocca "Bruni" e attende "Rossi".

L'output mostra in azione il meccanismo di Deadlock Detection del DBMS. Invece di rimanere bloccate indefinitamente, Postgres risolve automaticamente lo stallo circolare imponendo l'aborto di una delle due transazioni per sbloccare la situazione e far concludere con successo l'altra.

```

1 [DEADLOCK] --- INIZIO TEST DEADLOCK ---
2 [DEADLOCK] Tx1 ha bloccato matricola 101.
3 [DEADLOCK] Tx2 ha bloccato matricola 102.
4 [DEADLOCK] Tx2 prova a bloccare matricola 101...
5 [DEADLOCK] Tx1 prova a bloccare matricola 102...
6 [DEADLOCK] Tx1 ABORTITA per Deadlock!
7 [DEADLOCK] Tx2 ha completato entrambi gli update.

```

2.6 Scenario 6: Dirty Read

Questo scenario è stato introdotto per verificare lo scostamento tra lo standard SQL e Postgres. T1 effettua un aggiornamento di un record senza fare commit (procedendo poi con un rollback), mentre T2 tenta di leggere il record modificato.

Pur richiedendo esplicitamente a livello di driver il livello `READ UNCOMMITTED`, l'evidenza dei test dimostra che la lettura sporca non si verifica mai. T2 legge esclusivamente l'ultimo dato regolarmente confermato. Il DBMS analizzato converte infatti silenziosamente il livello in `READ COMMITTED`, escludendo a priori la possibilità di letture "sporche".

```

1 [DIRTY_READ] --- INIZIO TEST DIRTY READ (RICHIEDENDO READ_UNCOMMITTED) ---
2 [DIRTY_READ] T1 ha modificato il conteggio a 999 ma NON HA FATTO COMMIT.
3 [DIRTY_READ] T2 ha letto il conteggio: 1
4 [DIRTY_READ] DIRTY READ PREVENUTO! (Postgres infatti forza READ COMMITTED)
5 [DIRTY_READ] T1 ha fatto ROLLBACK.

```

3 Conclusioni

Gli esperimenti condotti hanno permesso di verificare empiricamente i concetti di controllo della concorrenza. La sperimentazione su PostgreSQL ha evidenziato come l'utilizzo del modello MVCC allontani parzialmente il DBMS dalle definizioni classiche. L'approccio di Postgres garantisce una prevenzione stringente delle anomalie (come i Phantom Read già in **REPEATABLE READ**), scaricando però a livello applicativo la necessità di intercettare e gestire tramite ulteriori tentativi le transazioni interrotte forzatamente per garantire la corretta serializzabilità.